STUDENT CAMPUS SERVICE MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

KAMALESHWARAN K

(241501079)

in partial fulfilment of the End Semester Examination for the course

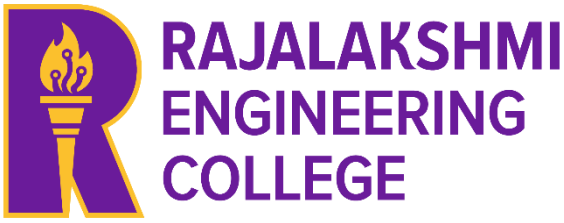
CB23F35 –

LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF ENGINEERING

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

OCTOBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled **"STUDENT CAMPUS SERVICE MANAGEMENT SYSTEM"** is the bonafide work of **KAMALESHWARAN K (241501079)**, who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year 2026–2027, Semester V, under my supervision.

SIGNATURE

Dr. M. Ayyadurai
HEAD OF THE DEPARTMENT
Department of AIML
Rajalakshmi Engineering College
Thandalam – 602 105

SIGNATURE

J. Sangeetha Archana
Asst Proffesor
Department of AIML
Rajalakshmi Engineering College

Submitted for the End Semester Examination (Individual Project) held on _____ 2026.

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I, KAMALESHWARAN K (2116241501079), hereby declare that the project report titled “STUDENT CAMPUS SERVICE MANAGEMENT SYSTEM”, submitted for the End Semester Examination of the course CB23F35 – Low-Code Application Development using Zoho Creator, is a record of original work carried out by me individually. The Zoho Creator application described in this report—including its data model, forms, reports, pages, Deluge scripts and workflows—was designed and developed by me. I further declare that any template, AI-assisted feature, code snippet or external resource used in this work has been acknowledged in the report.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. Every template, sample application, AI-assisted feature (such as Zia or the AI application builder), code snippet or external resource used in this work has been duly acknowledged in the report.

Place: Thandalam,Tamil nadu
Date:

SIGNATURE OF THE STUDENT
KAMALESHWARAN K
241501079

ACKNOWLEDGEMENT

I express my sincere gratitude to the management of Rajalakshmi Engineering College for providing the infrastructure and facilities required to complete this project.

I am grateful to the Principal of Rajalakshmi Engineering College for encouraging students to develop practical and industry-relevant applications.

I thank the Head of the Department of Artificial Intelligence and Machine Learning for the support provided during the course.

I sincerely thank the course faculty for guidance, feedback and suggestions during the design and development of this application.

I acknowledge Zoho Creator and its learning resources for providing the low-code platform used in this project. I also thank my family and friends for their support.

KAMALESHWARAN K

ABSTRACT

Colleges provide many day-to-day services to students, such as hostel maintenance, library support, transport passes, IT and Wi-Fi support, certificate requests and canteen or mess assistance. When these requests are collected on paper, through phone calls or in separate spreadsheets, they are easily lost, there is no common view of their status, and administrators cannot tell which requests are overdue or how satisfied students are with the service.

This project develops a Student Campus Service Management System using Zoho Creator. The application contains six related forms: Student Management, Staff Management, Service Category, Service Request, Payment Management and Feedback Management. Lookup fields connect the records, while field rules and Deluge scripts validate input and calculate the due date of each request from the SLA days of its service category. Workflows send acknowledgement, assignment and resolution emails, a scheduled workflow escalates overdue requests every morning, and an approval process controls certificate requests. A Zoho OAuth connection is used to raise high-priority alerts in Zoho Cliq and to read request data through the Creator REST API. Zia AI fields analyse the sentiment of feedback comments.

Role-based access is configured for Administrator, Service Manager, Service Staff and Student (portal) users, and students see only their own records. A dashboard with summary tiles and four charts, together with six reports, gives a consolidated view of requests, staff workload and fee collection. The application is available through the browser, the customer portal and the Zoho Creator mobile app. The result is a single low-code application that replaces scattered registers with traceable, role-controlled and automated campus service handling.

Keywords: Campus Service Management; Zoho Creator; Low-Code Development; Deluge Scripting; Workflow Automation; Role-Based Access Control; Zia AI

PROJECT AT A GLANCE

Attribute	Details
Project Title	Student Campus Service Management System
Application Domain	Education / Campus Administration and Student Services
Student Name & Register No.	Kamaleshwaran K – 241501079
Zoho Creator Edition	Zoho Creator (trial / evaluation edition)
Application Name (Link Name)	Student_Campus_Service_Management_System
Published / Live URL	Provided during the demonstration (trial account expired)
Demo Credentials (per role)	Not included in the report; shared securely during the demonstration
Number of Forms / Subforms	6 / 0
Number of Reports (by type)	6 – List (4), Kanban (1), Summary (1)
Number of Pages / Dashboards	1 dashboard page (4 summary tiles and 4 charts)
Number of Workflows	8 – 5 form workflows, 1 schedule, 1 approval process, 1 Blueprint
Deluge Functions / Custom Actions	6 scripts (DS-01 to DS-06)
Integrations	Zoho OAuth connections: Zoho Cliq (invokeurl) and Zoho Creator REST API v2 (tested in Postman)
Roles & Profiles Defined	Administrator, Service Manager, Service Staff, Student (portal)
Environments Used	Development, Stage, Production
Date of Production Deployment	October 2026

TABLE OF CONTENTS

ABSTRACT v
PROJECT AT A GLANCE vi
LIST OF TABLES x
LIST OF FIGURES xi
LIST OF ABBREVIATIONS xii
INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 1
1.4 Scope of the Project 2
1.5 Justification for a Low-Code Approach 2
1.6 Organisation of the Report 2
LITERATURE SURVEY AND SYSTEM STUDY 3
2.1 Study of the Existing System 3
2.2 Limitations of the Existing System 3
2.3 Low-Code versus Traditional Development 4
2.4 Literature Survey 4
2.5 Emerging Trends in Low-Code Development 5
2.6 Proposed System 5
REQUIREMENT ANALYSIS 7
3.1 Stakeholders and User Roles 7
3.2 Functional Requirements 7
3.3 Non-Functional Requirements 8
3.4 Hardware and Software Requirements 8
3.5 Use Case Model 9
SYSTEM DESIGN 11
4.1 System Architecture 11
4.2 Data Model 12
4.3 Form Design 13
4.4 Process and Workflow Design 14
4.5 User Interface and Navigation Design 14
4.6 Access Control Design 14
IMPLEMENTATION 16
5.1 Application Setup 16
5.2 Forms and Data Capture 16

5.3 Reports 18
5.4 Pages and Dashboards 18
5.5 Deluge Scripting 19
5.6 Workflows and Automation 21
5.7 Integrations 22
5.8 User Management and Role-Based Access 24
5.9 AI and Analytics Features 25
5.10 Pro-Code Extension 26
5.11 Mobile and Portal Access 26
TESTING AND DEBUGGING 27
6.1 Testing Strategy 27
6.2 Test Cases 27
6.3 Debugging Deluge Scripts 28
6.4 Defect Log 29
6.5 User Acceptance Testing 29
DEPLOYMENT, SECURITY AND GOVERNANCE 30
7.1 Environments and Deployment Strategy 30
7.2 Performance Optimisation 30
7.3 Security Features 31
7.4 Governance and Administration 32
RESULTS AND DISCUSSION 33
8.1 Application Output 33
8.2 Achievement of Objectives 34
8.3 Comparison with the Existing System 35
8.4 Limitations of the Low-Code Approach 35
8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum 35
CONCLUSION AND FUTURE ENHANCEMENTS 36
9.1 Conclusion 36
9.2 Future Enhancements 36
9.3 Learning Outcomes and Career Relevance 36
REFERENCES 37
APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST 38
APPENDIX B – COURSE OUTCOME MAPPING 40
APPENDIX C – COMPLETE DELUGE SOURCE CODE 41
APPENDIX D – USER MANUAL 44
D.1 Accessing the Application 44
D.2 Instructions for Students 44

D.3 Instructions for Service Staff 44
D.4 Instructions for the Administrator 44

LIST OF TABLES

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application 4
Table 2.2: Summary of Literature Survey 4
Table 3.1: Stakeholders and User Roles 7
Table 3.2: Functional Requirements 7
Table 3.3: Non-Functional Requirements 8
Table 3.4: Hardware and Software Requirements 8
Table 3.5: Use Case Description – Submit and Resolve a Service Request 10
Table 4.1: Entities and Relationships 12
Table 4.2: Field Specification – Student Campus Service Management Forms 13
Table 4.3: Role–Permission Matrix 14
Table 5.1: Reports Implemented 18
Table 5.2: Deluge Scripts Implemented 19
Table 5.3: Workflows Implemented 21
Table 5.4: Integrations Implemented 22
Table 6.1: Test Cases 27
Table 6.2: Defect Log 29
Table 6.3: User Acceptance Feedback 29
Table 7.1: Deployment Checklist 30
Table 7.2: Performance Optimisation Measures 31
Table 7.3: Security Controls Implemented 31
Table 8.1: Mapping of Objectives to Outcomes 34
Table 8.2: Existing System versus Proposed Application 35
Table A.1: Feature Coverage Checklist 38
Table B.1: Mapping of Course Outcomes to Report Chapters 40

LIST OF FIGURES

Figure 2.1: As-Is Process Flow of the Existing System 3
Figure 3.1: Use Case Diagram of the Student Campus Service Management System. 9
Figure 4.1: System Architecture of the Student Campus Service Management System 11
Figure 4.2: Entity–Relationship Diagram of the Student Campus Service Management System 12
Figure 4.3: Workflow / Process Flow for Service Request Handling and Notification 14
Figure 4.4: Navigation Structure and Wireframe of the Campus Service Dashboard 14
Figure 5.1: Application Home and Navigation Sections 16
Figure 5.2: Student Management Form in the Form Builder 17
Figure 5.3: Service Request Form in the Form Builder 17
Figure 5.4: Service Request Form in Live Mode 17
Figure 5.5: All Service Requests Report 18
Figure 5.6: Campus Service Dashboard – Summary Tiles 19
Figure 5.7: Campus Service Dashboard – Charts 19
Figure 5.8: Workflow Configuration for Request Notifications 22
Figure 5.9: Connection Setup in Zoho Creator 23
Figure 5.10: Postman Request and JSON Response from the Creator REST API 23
Figure 5.11: Roles and Users Configuration 24
Figure 5.12: Permission Profile Configuration for Service Staff 25
Figure 5.13: Zia AI Fields and Builder Assistant in Feedback Management 25
Figure 5.14: Student Portal Sign-In and Mobile View of My Requests 26
Figure 6.1: Deluge Debug Output for the Due-Date Script 29
Figure 7.1: Environments and Publish to Production 30
Figure 7.2: Audit Trail and Administration Settings 32
Figure 8.1: Scenario Step 1 – Student Submits a Service Request in the Portal 33
Figure 8.2: Scenario Step 2 – Administrator Assigns the Request to Staff 33
Figure 8.3: Scenario Step 3 – Resolution Email Received by the Student 34

LIST OF ABBREVIATIONS

Abbreviation	Expansion
AI	Artificial Intelligence
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
Deluge	Data Enriched Language for the Universal Grid Environment
ER	Entity–Relationship
FR / NFR	Functional / Non-Functional Requirement
HTML	HyperText Markup Language
JSON	JavaScript Object Notation
OAuth	Open Authorization
RBAC	Role-Based Access Control
REST	Representational State Transfer
SLA	Service Level Agreement
UAT	User Acceptance Testing
UI	User Interface
URL	Uniform Resource Locator
Zia	Zoho's AI assistant (Zoho Intelligence Assistant)
Zoho Creator	Low-code application development platform

CHAPTER 1
INTRODUCTION

1.1 Background

The Student Campus Service Management System is a business application for a college campus that must handle many kinds of student service requests in an organised way. Students need help with hostel maintenance, library services, transport passes, IT and Wi-Fi problems, certificates and canteen or mess issues. Each service is owned by a department or staff member, and each has its own expected turnaround time. The application uses Zoho Creator forms for data entry, lookup fields to connect records, reports and a dashboard for monitoring, workflows for notifications and an AI-assisted feedback form. It can be accessed from a browser, the student portal and the mobile app, according to the permissions of each user.

1.2 Problem Statement

A campus service desk must receive each request, identify its category and priority, assign it to the right staff member, track its progress against a deadline, collect any service charge and finally capture the student's feedback. When these steps are maintained on paper slips, registers or disconnected spreadsheets, the following problems arise.

- Requests may be lost, duplicated or forgotten because there is no single record of every request.
- Students cannot see the status of their requests and must visit or call the office repeatedly.
- Administrators cannot easily identify overdue requests, overloaded staff or the fees still pending.
- Feedback is not captured in a structured way, so service quality cannot be measured.

1.3 Objectives

- To create related forms for students, service staff, service categories, service requests, payments and feedback.
- To automate input validation, due-date calculation and selected notifications using Deluge scripts and workflows.

- To provide reports and a dashboard for reviewing request status, staff workload, fee collection and student feedback.
- To integrate the application with Zoho Cliq and the Creator REST API through secure OAuth connections, and to apply Zia AI to analyse feedback comments.
- To apply role-based access for Administrator, Service Manager, Service Staff and Student users and verify it through testing.
- To deploy the application through Development, Stage and Production environments with security and governance controls.

1.4 Scope of the Project

In scope: student, staff, service category, service request, payment and feedback records; lookup relationships; validation; due-date calculation from SLA days; reports; dashboard and charts; email and Zoho Cliq notifications; scheduled overdue escalation; approval of certificate requests; role-based access and a student portal; Zia sentiment analysis on feedback; mobile access; and deployment across environments.

Out of scope: online payment-gateway processing, biometric or ID-card integration, integration with the college ERP or examination system, and production-scale performance testing with very large datasets.

1.5 Justification for a Low-Code Approach

Zoho Creator is suitable for this application because its visual form builder, lookup fields, reports, pages, workflows and Deluge scripting allow related business records and automation to be built quickly without developing a separate database, back-end server and user interface. Built-in hosting, role and profile management, a customer portal, a mobile app and connections to other Zoho services reduce infrastructure effort, so more time can be spent on the service-desk process itself.

1.6 Organisation of the Report

Chapter 2 reviews the existing system and literature. Chapter 3 presents the requirement analysis and Chapter 4 the system design. Chapter 5 details the implementation, Chapter 6 testing, Chapter 7 deployment and security, Chapter 8 results and Chapter 9 the conclusion.

CHAPTER 2
LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

In a manual process, a student visits the hostel office, library, transport office or administrative office and writes a complaint slip or fills a paper form. A clerk notes the request in a register or spreadsheet and passes it on verbally or by phone to the responsible staff member. The staff member acts on it, but there is no formal status update, so the student returns repeatedly to ask whether the request has been handled. Fees for services such as transport passes or certificates are recorded separately, and feedback is rarely collected.

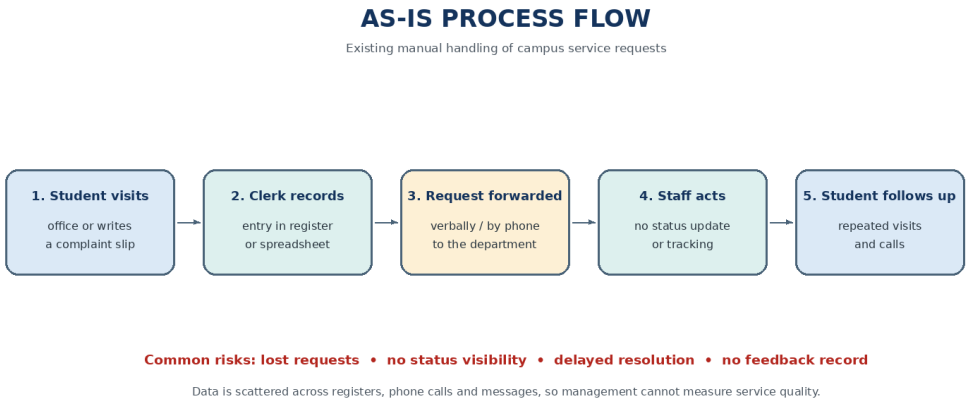


Figure 2.1: As-Is Process Flow of the Existing System

2.2 Limitations of the Existing System

- Records are scattered across registers, slips and spreadsheets, making retrieval and follow-up slow.
- There is no automatic acknowledgement, due date or reminder, so urgent requests can be delayed without anyone noticing.
- Students have no visibility of request status, and administrators have no summary of workload, overdue items or feedback.
- Fee collection and service quality cannot be reviewed because data is not stored in a structured and connected form.

2.3 Low-Code versus Traditional Development

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Development time	Requires custom UI, database, backend, testing and hosting setup.	Visual form/report builders and built-in hosting shorten initial development; actual time depends on scope.
Skills required	Programming, database design, deployment and maintenance skills.	Visual configuration plus Deluge and platform knowledge.
Hosting & infrastructure	Application server, database and deployment pipeline must be maintained.	Hosting is provided through the Zoho Creator service.
Mobile application	Usually requires responsive web design or separate mobile development.	Creator provides platform-supported mobile access for the same application.
Security & access control	Must be designed and implemented in application code and infrastructure.	Built-in roles, profiles and portal permissions are available but must be configured and tested.
Maintenance & change	Code and infrastructure changes may require developer effort.	Many form and report changes can be made visually; scripts still need maintenance.
Customisation limits	Broad control over architecture and runtime.	Bounded by Creator features, limits and supported integrations.
Cost	Development, hosting, monitoring and maintenance costs vary.	Subscription/edition and usage costs depend on the selected plan.

2.4 Literature Survey

Table 2.2: Summary of Literature Survey

S. No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
1	Zoho Corporation, Zoho Creator Help	Forms and fields documentation	Creator forms provide configurable field types, validation and data capture.	Supports the six form modules and field rules.
2	Zoho Corporation, Zoho Creator Help	Lookup fields and relationships	Lookup fields connect records across forms.	Supports student–request, category–request and request–payment relationships.

S. No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
3	Zoho Corporation, Deluge Help	Deluge scripting documentation	Deluge provides control flow, record access, sendmail and invokeurl for integrations.	Supports validation, due-date calculation, notifications and API calls.
4	Zoho Corporation, Zoho Creator Help	Workflows, approvals and Blueprint	Workflows run actions on record events or schedules; approvals and Blueprint control process stages.	Supports acknowledgement emails, escalation and the request lifecycle.
5	Zoho Corporation, Zoho Creator Help	Connections and REST API v2	OAuth connections authorise calls to Zoho services and external APIs without storing passwords in scripts.	Supports the Zoho Cliq alert and the Postman API test.
6	Sahay et al., 2020	Supporting the understanding and comparison of low-code development platforms	Compares features of low-code platforms and highlights visual modelling, integration and deployment support.	Justifies the choice and evaluation of a low-code platform.
7	OWASP Foundation	Application Security Verification Standard (ASVS)	Security should be assessed through access control, input validation and secure handling of data.	Informs permission checks and validation testing.

2.5 Emerging Trends in Low-Code Development

Low-code platforms increasingly include AI-assisted application generation, visual workflow builders, ready-made connectors and AI services such as sentiment analysis and prediction. In this project, an AI assistant helped to build part of the form structure, and Zia sentiment and keyword fields are used to analyse the text of student feedback. AI-generated configuration must always be reviewed against the real requirements, which is why every Zia-generated field and script in this application was checked and tested before use. Integration with collaboration tools such as Zoho Cliq is another trend that turns a data-entry application into a live service desk.

2.6 Proposed System

The proposed system uses six connected forms as the central data model. Student, staff and service category information is maintained once and referenced through lookup

fields in the Service Request form. Payments and feedback are linked to the request they belong to. Validation and Deluge scripts calculate due dates and block invalid input; workflows acknowledge requests, notify staff, invite feedback and escalate overdue requests; and an approval process controls certificate requests. Reports and a dashboard summarise requests, staff workload and fees, and the student portal lets students submit and track their own requests. The effectiveness of each workflow is demonstrated using test records and delivered emails.

CHAPTER 3
REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

Table 3.1: Stakeholders and User Roles

Role	Description	Key Responsibilities in the Application
Administrator	Application owner or campus service administrator	Manage all records, users, roles, workflows, configuration and reports.
Service Manager	Head of a service department	Review and assign requests, monitor the dashboard and approve certificate requests.
Service Staff	Staff who resolve requests (warden, librarian, IT support, transport and office staff)	View assigned requests, update status and resolution, and record payments for chargeable services.
Student	Student who needs a campus service (portal user)	Submit requests, track own requests, view own payments and give feedback.

3.2 Functional Requirements

Table 3.2: Functional Requirements

FR ID	Requirement	Role(s)	Priority
FR-01	The system shall allow authorised users to create and maintain student and service staff records with mandatory contact fields.	Administrator, Manager	High
FR-02	The system shall maintain service categories with owner department, SLA days and fee.	Administrator	High
FR-03	The system shall allow students to submit service requests linked to their own record and a service category.	Student, Staff	High
FR-04	The system shall calculate the due date of a request from its request date and the SLA days of its category.	System	High
FR-05	The system shall allow requests to be assigned to staff and tracked through New, Assigned, In Progress, Resolved and Closed.	Administrator, Manager, Staff	High
FR-06	The system shall record payments for chargeable services and report pending payments.	Staff, Administrator	Medium
FR-07	The system shall send acknowledgement, assignment and resolution emails when workflow conditions are met.	System workflow	High

FR ID	Requirement	Role(s)	Priority
FR-08	The system shall escalate requests that pass their due date and alert Zoho Cliq for high-priority requests.	System workflow	Medium
FR-09	The system shall capture ratings and comments and analyse comment sentiment using Zia.	Student, System	Medium
FR-10	The system shall display a dashboard with summary tiles and charts on requests, staff workload and fees.	Administrator, Manager	Medium
FR-11	The system shall restrict access to forms, reports and records according to the user's role and profile.	All roles	High

3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

NFR ID	Category	Requirement
NFR-01	Performance	Reports and the dashboard should load within a few seconds for the expected number of campus records.
NFR-02	Security	Users shall access only forms, reports and records allowed by their assigned role and profile; connections use minimum OAuth scopes.
NFR-03	Usability	Forms should use clear labels, required-field indicators and understandable validation messages on desktop and mobile.
NFR-04	Availability	Availability depends on the Zoho Creator service and the user's internet connection.
NFR-05	Maintainability	Forms, fields, scripts and workflows should use meaningful names and be documented for future changes.
NFR-06	Auditability	Record and configuration changes should be traceable through the audit trail.

3.4 Hardware and Software Requirements

Table 3.4: Hardware and Software Requirements

Category	Specification
Platform	Zoho Creator (trial / evaluation edition)
Scripting language	Deluge scripting
Browser	Current version of Chrome, Edge or Firefox
API testing tool	Postman (to test the Creator REST API v2 with the OAuth connection)

Category	Specification
Other Zoho services	Zoho Cliq (alerts), Zoho OAuth connections, Zia AI fields
Mobile	Zoho Creator mobile app (Android / iOS) and student portal on a mobile browser
Client hardware	Intel Core i3/i5 or above, 8 GB RAM, internet connection

3.5 Use Case Model

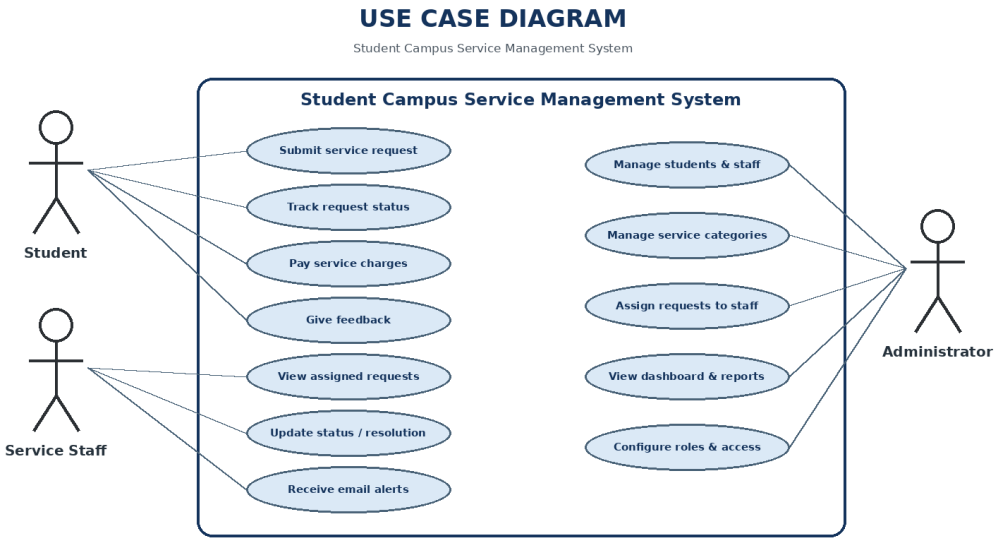


Figure 3.1: Use Case Diagram of the Student Campus Service Management System

Table 3.5: Use Case Description – Submit and Resolve a Service Request

Use Case ID	UC-01
Use Case Name	Submit and Resolve a Service Request
Actor(s)	Student, Administrator / Service Manager, Service Staff
Pre-condition	The student has a portal account; service categories and staff records exist.
Main Flow	1. Student signs in to the portal and opens Service Request. 2. Student selects the service category, enters subject, description and priority, and submits. 3. The system sets the due date and sends an acknowledgement email. 4. Administrator assigns a staff member; the staff member is notified. 5. Staff updates the status to In Progress and then Resolved. 6. The student receives a resolution email and submits feedback.
Alternate Flow	If a required field or lookup value is missing, the form shows a validation message and the request is not saved. If the student email is missing, the acknowledgement email is not sent. High-priority requests also raise a Zoho Cliq alert.
Post-condition	The request, its status history and the feedback are saved and visible in the reports and dashboard.
Related FR	FR-03, FR-04, FR-05, FR-07, FR-09

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

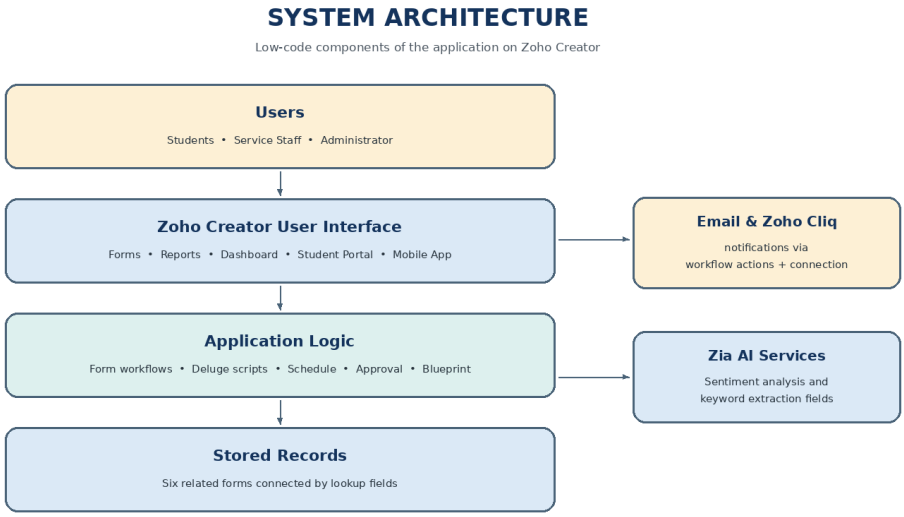


Figure 4.1: System Architecture of the Student Campus Service Management System

Users access the Zoho Creator application through its user interface, the student portal or the mobile app. Forms capture and store student, staff, category, request, payment and feedback records, and lookup fields connect the related records. Reports and the dashboard read the stored data for review. Form workflows, Deluge scripts, a schedule, an approval process and a Blueprint implement validation, calculation and notifications. Connections authorise calls to Zoho Cliq and the Creator REST API, and Zia AI fields analyse feedback text. Access to forms, reports and actions is controlled by roles and profiles.

4.2 Data Model

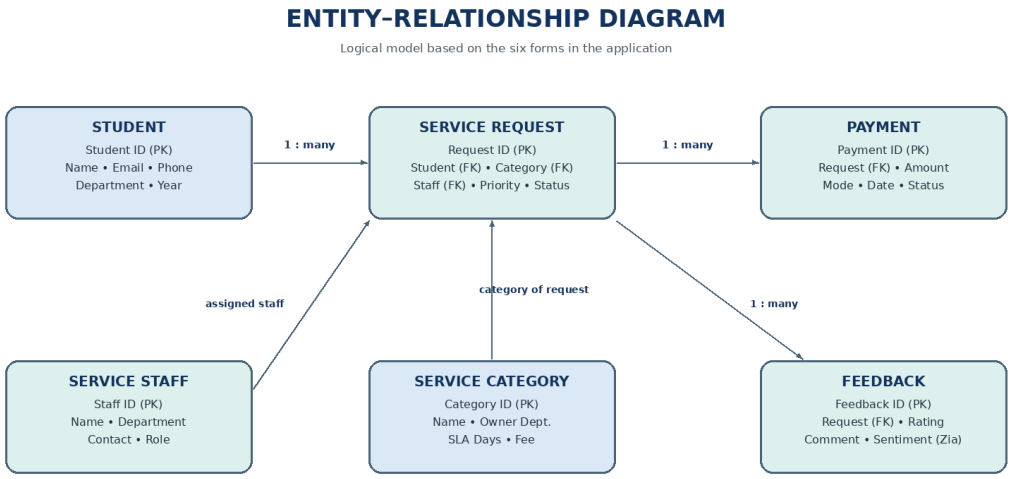


Figure 4.2: Entity–Relationship Diagram of the Student Campus Service Management System

Table 4.1: Entities and Relationships

Entity (Form)	Key Fields	Related To	Relationship Type	Implemented Using
Student Management	Student ID, Student Name, Email	Service Request	One-to-Many	Student lookup in Service Request
Service Category	Category ID, Category Name, SLA Days, Fee	Service Request	One-to-Many	Category lookup in Service Request
Staff Management	Staff ID, Name, Department, Contact	Service Request	One-to-Many	Assigned Staff lookup in Service Request
Service Request	Request ID, Subject, Priority, Status, Due Date	Payment Management	One-to-Many	Request lookup in Payment
Service Request	Request ID, Subject, Priority, Status, Due Date	Feedback Management	One-to-Many	Request lookup in Feedback

No subform is used. Payments and feedback are stored as separate forms because a request can have several payments and each feedback entry needs its own report and permissions; a subform would have hidden these records from reports.

4.3 Form Design

Table 4.2: Field Specification – Student Campus Service Management Forms

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Student ID	Student_ID	Auto Number	Auto	Prefix STU-	Student identity
Student Name	Student_Name	Name	Yes	Required	Student identity
Email	Email	Email	Yes	Valid, no duplicates	Used for notifications
Department	Department	Dropdown	Yes	Required	AIML, CSE, ECE, IT, MECH
Category Name	Category_Name	Single Line	Yes	Required	Service master
SLA Days	SLA_Days	Number	Yes	Must be >= 1	Used for due date
Service Fee	Service_Fee	Currency	No	Must be >= 0	Chargeable services
Student	Student	Lookup	Yes	Student Management	Request owner
Service Category	Service_Category	Lookup	Yes	Service Category	Type of service
Subject	Subject	Single Line	Yes	Required	Request title
Priority	Priority	Radio	Yes	Default: Medium	High / Medium / Low
Request Date	Request_Date	Date	Yes	Default: today	Cannot be blank
Due Date	Due_Date	Date	No	Set by DS-01	Request date + SLA days
Assigned Staff	Assigned_Staff	Lookup	No	Staff Management	Set by administrator
Status	Status	Dropdown	Yes	Default: New	Managed by Blueprint
Amount	Amount	Currency	Yes	Must be >= 0	Payment value
Rating	Rating	Radio	Yes	1 to 5	Feedback score
Comment Sentiment	Comment_Sentiment	Zia AI field	No	From Comment	Positive / Neutral / Negative

4.4 Process and Workflow Design

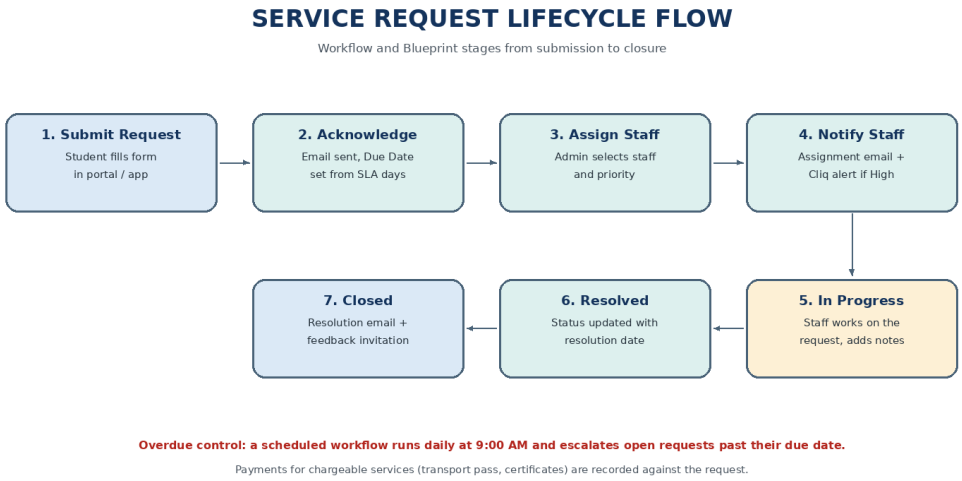


Figure 4.3: Workflow / Process Flow for Service Request Handling and Notification

4.5 User Interface and Navigation Design

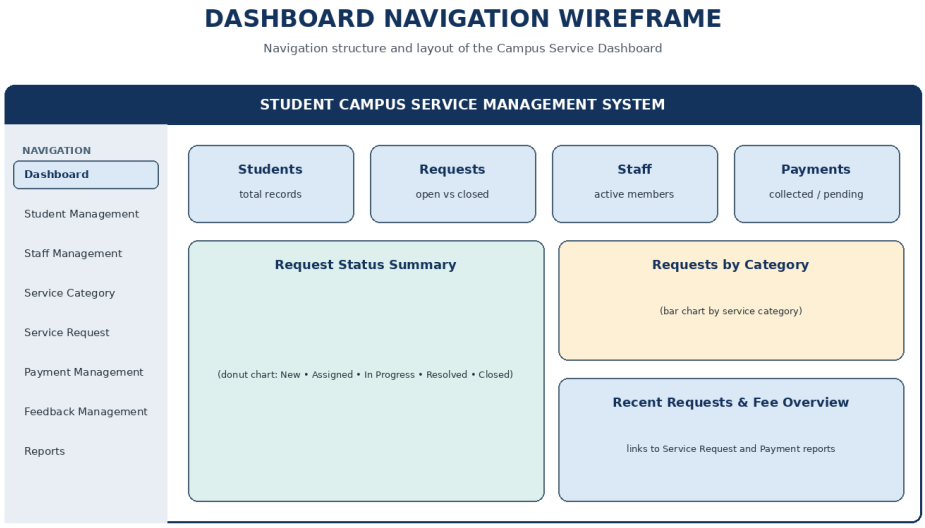


Figure 4.4: Navigation Structure and Wireframe of the Campus Service Dashboard

4.6 Access Control Design

Table 4.3: Role–Permission Matrix

Component	Administrator	Manager	Staff	Student Portal
Form: Student Management	A E V D	A E V	V	—
Form: Staff Management	A E V D	A E V	V	—
Form: Service Category	A E V D	E V	V	V

Component	Administrator	Manager	Staff	Student Portal
Form: Service Request	A E V D	A E V	E V (assigned)	A V (own)
Form: Payment Management	A E V D	A E V	A V	V (own)
Form: Feedback Management	A E V D	V	V	A V (own)
Report: All Service Requests	V	V	V (assigned)	V (own)
Report: Pending Payments	V	V	V	—
Page: Campus Service Dashboard	V	V	—	—

A = Add, E = Edit, V = View, D = Delete. “(own)” and “(assigned)” indicate record-level criteria.

CHAPTER 5
IMPLEMENTATION

5.1 Application Setup

The application “Student Campus Service Management System” was created in Zoho Creator with the link name Student_Campus_Service_Management_System. Its navigation is organised into sections for the six data-entry forms, the reports and the Campus Service Dashboard. The home page offers quick links to add records and to open the main reports, and the project uses Creator’s visual form builder, workflows, Deluge scripting, connections and Zia features, as described in the following sections.

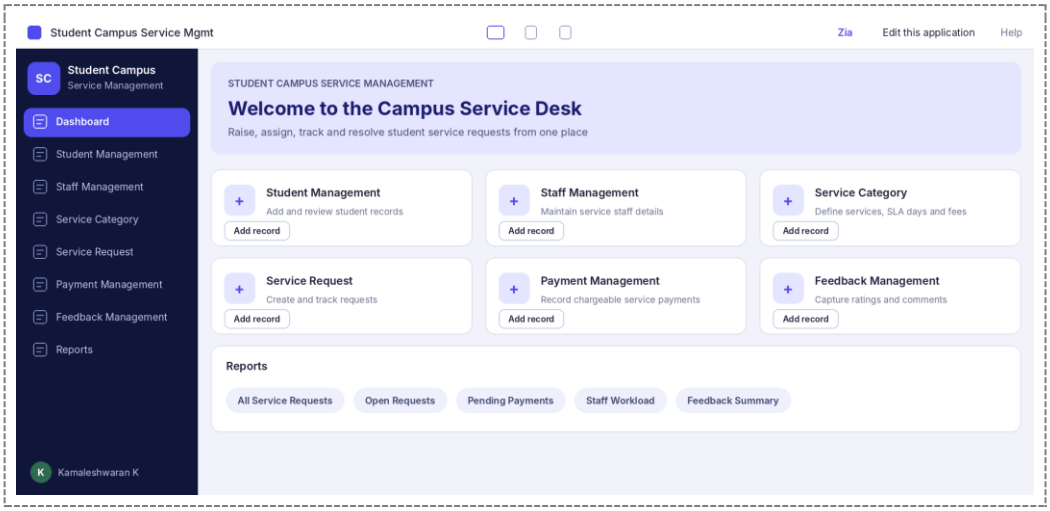


Figure 5.1: Application Home and Navigation Sections

5.2 Forms and Data Capture

The application contains six main forms. Student Management stores identity and contact details; Staff Management stores the service staff who resolve requests; Service Category defines each service with its owner department, SLA days and fee; Service Request records every request with its student, category, priority, dates, assigned staff and status; Payment Management records payments for chargeable services against a request; and Feedback Management stores the rating and comment for a request. Across the forms, more than ten distinct field types are used, including Auto Number, Name, Single Line, Multi Line, Email, Phone, Number, Currency, Date, Dropdown, Radio, Checkbox, Lookup, File Upload, Image and Zia AI fields. Required fields and unique-value checks prevent incomplete or duplicate records.

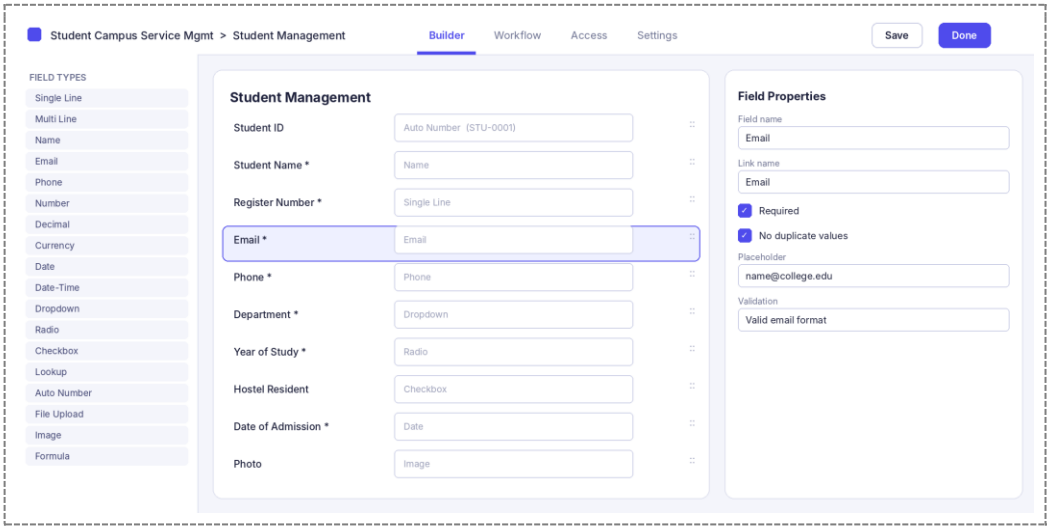


Figure 5.2: Student Management Form in the Form Builder

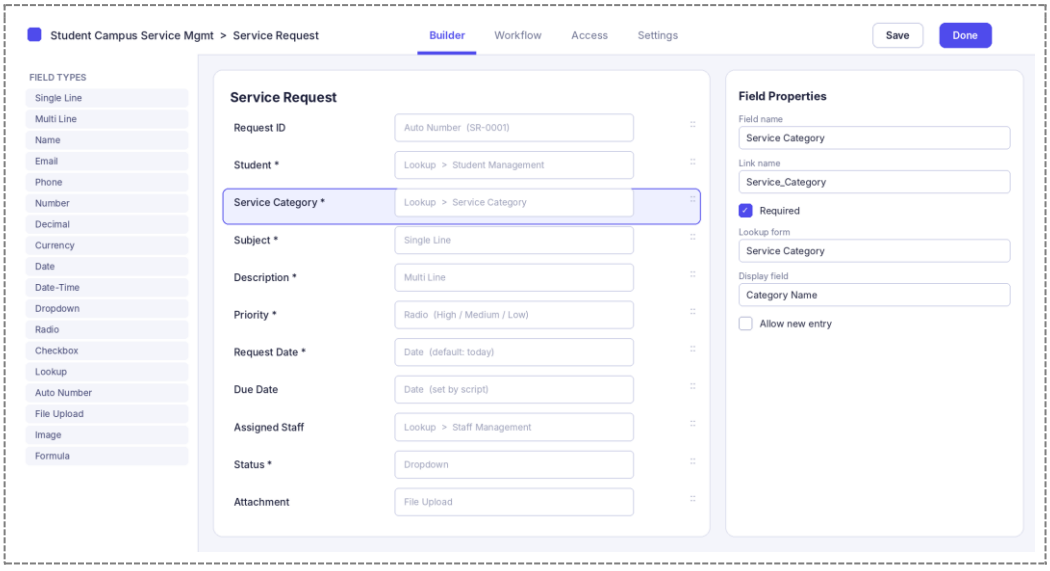


Figure 5.3: Service Request Form in the Form Builder

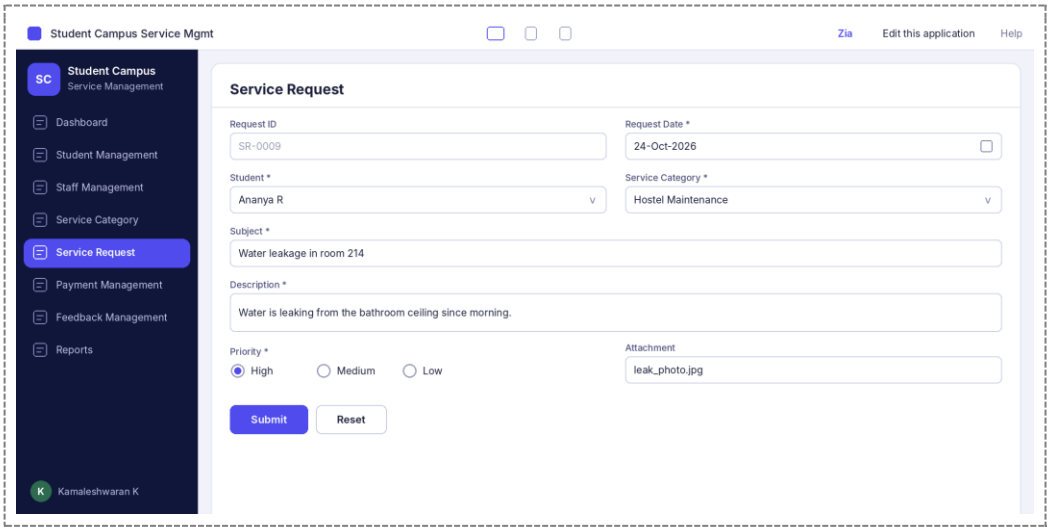


Figure 5.4: Service Request Form in Live Mode

5.3 Reports

Table 5.1: Reports Implemented

Report Name	Type	Source Form	Filter / Grouping / Sorting	Custom Actions	Roles
All Service Requests	List	Service Request	Filters: Status, Priority, Category, Date; sort by request date	Open / edit record, export	Administrator, Manager, Staff (assigned)
Open Requests	List	Service Request	Criteria: Status is not Resolved or Closed; sort by due date	Assign staff	Administrator, Manager
Request Board	Kanban	Service Request	Grouped by Status	Drag to change status (Blueprint)	Administrator, Manager, Staff
Pending Payments	List	Payment Management	Filter: Status = Pending; sort by date	Open payment record	Administrator, Manager, Staff
Staff Workload	Summary	Service Request	Grouped by Assigned Staff and Status; count of records	Drill down to records	Administrator, Manager
Feedback Summary	List	Feedback Management	Filter by Rating and Sentiment	Open feedback record	Administrator, Manager

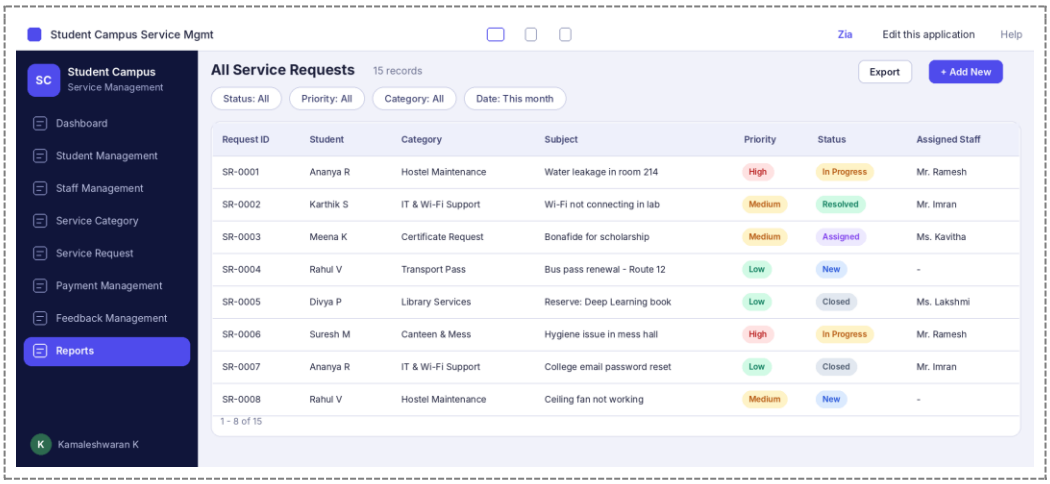


Figure 5.5: All Service Requests Report

5.4 Pages and Dashboards

The Campus Service Dashboard is a page built with the UI builder. It shows summary tiles for total students, service staff, service categories and total requests, together with open requests, resolved or closed requests, fees collected and pending fees. Below the

tiles, quick actions open the most common forms. Four charts, created from the Service Request and Payment forms, show requests by category (bar), requests by status (donut), monthly requests (line) and open requests by staff (horizontal bar). Dashboard values were compared with the underlying reports after records were added or edited.

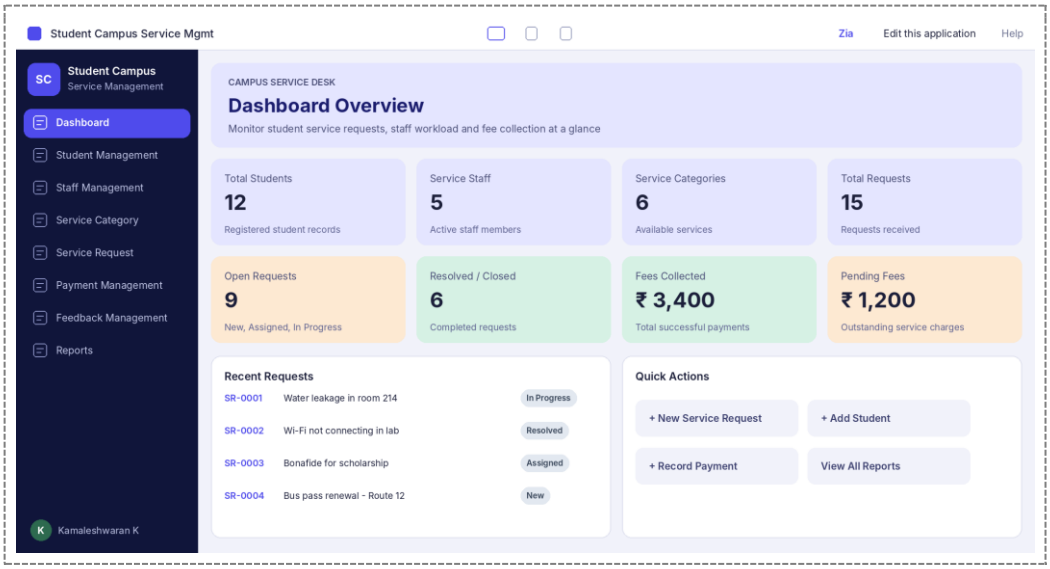


Figure 5.6: Campus Service Dashboard – Summary Tiles

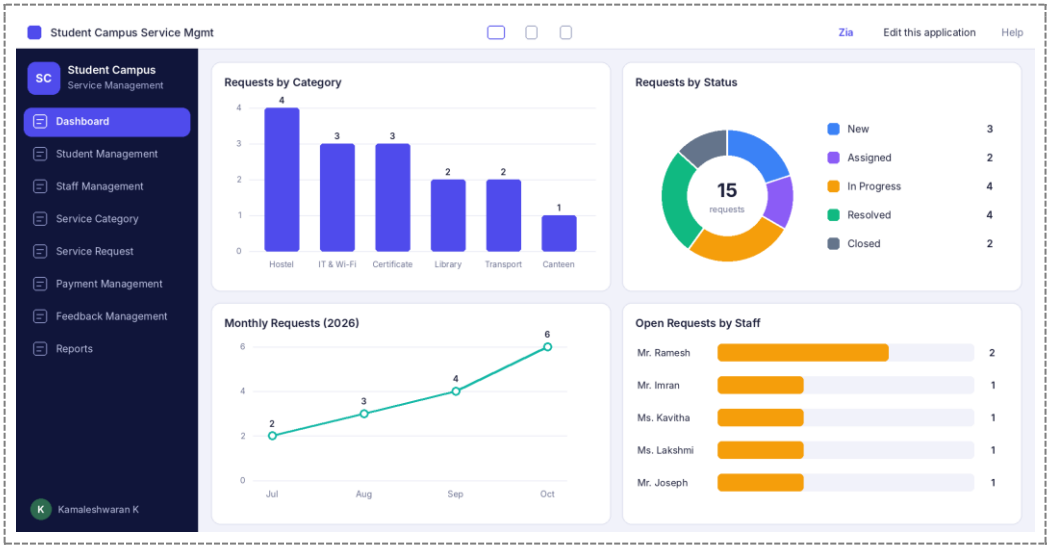


Figure 5.7: Campus Service Dashboard – Charts

5.5 Deluge Scripting

Table 5.2: Deluge Scripts Implemented

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-01	Service Request → On Add → On Validate	Validate required values and set the due date from the category SLA days.	criteria fetch, if conditions, date functions, alert, cancel submit

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-02	Service Request → On Add → On Success	Send an acknowledgement email to the student.	lookup fetch, if conditions, sendmail
DS-03	Service Request → On Edit → On Success	Notify the assigned staff member when a request is assigned.	condition on field change, sendmail
DS-04	Service Request → On Edit → On Success	Send resolution email with a feedback link when Status becomes Resolved.	lookup fetch, if conditions, sendmail
DS-05	Service Request → On Add → On Success	Post a Zoho Cliq alert for high-priority requests.	Map, invokeurl, connection, info
DS-06	Schedule: daily at 9:00 AM	Email the administrator about requests past their due date.	for each, criteria fetch, date comparison, sendmail

Code 5.1: Validation and Due-Date Calculation Script (DS-01)

```
// On Add > On Validate of Service Request
cat = Service_Category[ID == input.Service_Category];
if(input.Request_Date == null)
{
    alert "Request Date is required";
    cancel submit;
}
if(cat.SLA_Days == null || cat.SLA_Days < 1)
{
    alert "SLA days are not defined for this service category";
    cancel submit;
}
input.Due_Date = input.Request_Date.addDay(cat.SLA_Days.toLong());
```

Code 5.2: Acknowledgement Email Script (DS-02)

```
// On Add > On Success of Service Request
stu = Student_Management[ID == input.Student];
cat = Service_Category[ID == input.Service_Category];
if(stu.Email != "")
{
    sendmail
    [
        from :zoho.adminuserid
        to :stu.Email
        subject :"Request " + input.Request_ID + " received"
        message :"Dear " + stu.Student_Name + ", your " +
cat.Category_Name + " request is registered. Due date: " +
input.Due_Date
    ]
}
```

The validation script checks that the request date and the SLA days are acceptable before the record is saved. The due date is then derived from the category so that

students and staff see the same deadline. Email scripts fetch the saved student and category records through lookup fields and send a message only when a valid student email exists. The complete source code of all six scripts is given in Appendix C.

5.6 Workflows and Automation

Table 5.3: Workflows Implemented

Workflow Name	Type	Trigger / Event	Condition	Action(s)	Script Ref.
Set Due Date from SLA	Form workflow	On Add – On Validate	Request Date and SLA days available	Calculate Due Date	DS-01
Request Acknowledgement Email	Form workflow	On Add – On Success	Student email available	Send acknowledgement email	DS-02
Staff Assignment Notification	Form workflow	On Edit – On Success	Assigned Staff set and Status is Assigned	Email assigned staff	DS-03
Resolution & Feedback Invite	Form workflow	On Edit – On Success	Status becomes Resolved	Email student with feedback link	DS-04
High Priority Cliq Alert	Form workflow	On Add – On Success	Priority is High	Post alert to Zoho Cliq	DS-05
Overdue Request Escalation	Schedule	Daily at 9:00 AM	Due Date earlier than today and not Resolved or Closed	Notify administrator	DS-06
Certificate Approval	Approval process	On Add	Category is Certificate Request	Manager approves or rejects	—
Request Lifecycle	Blueprint	Status transitions	New → Assigned → In Progress → Resolved → Closed	Controls allowed status changes	—



Figure 5.8: Workflow Configuration for Request Notifications

Code 5.3: Overdue Escalation Script (DS-06)

```
// Schedule: Overdue Request Escalation - daily at 9:00 AM
for each rec in Service_Request[Due_Date < zoho.currentdate && Status
!= "Resolved" && Status != "Closed"]
{
    sendmail
    [
        from :zoho.adminuserid
        to :zoho.adminuserid
        subject :"Overdue request " + rec.Request_ID
        message :"Request " + rec.Request_ID + " (" + rec.Subject + ")
is past its due date " + rec.Due_Date
    ]
}
```

5.7 Integrations

Table 5.4: Integrations Implemented

Integration	Service / Endpoint	Method	Authentication	Data Exchanged / Purpose
Zoho Cliq alert	Cliq channel message API (channel: campusservices)	Deluge invokeurl, POST	Zoho OAuth connection cliq_alert_conn (ZohoCliq.Webhooks .CREATE)	Posts request ID and subject for high-priority requests
Creator REST API v2	report/All_Service_Requests	GET (tested in Postman)	Zoho OAuth connection creator_api_conn (ZohoCreator.report. READ)	Returns request records as JSON for external use
Email notifications	Zoho Creator workflow action	Deluge sendmail	Configured sender account	Acknowledgement, assignment, resolution and overdue emails

Two connections were created under Settings → Connections using the Zoho OAuth service. Each connection stores only the scopes it needs, so no password or token is written in any script. The Cliq connection is referenced by name in the invokeurl call of DS-05, and the Creator API connection authorised the Postman request that returned the JSON response shown in Figure 5.10.

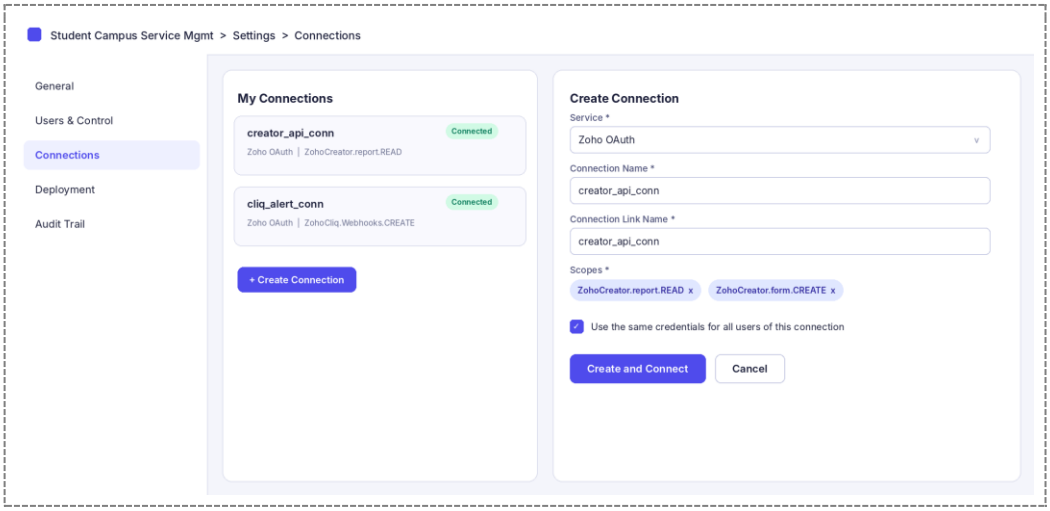


Figure 5.9: Connection Setup in Zoho Creator

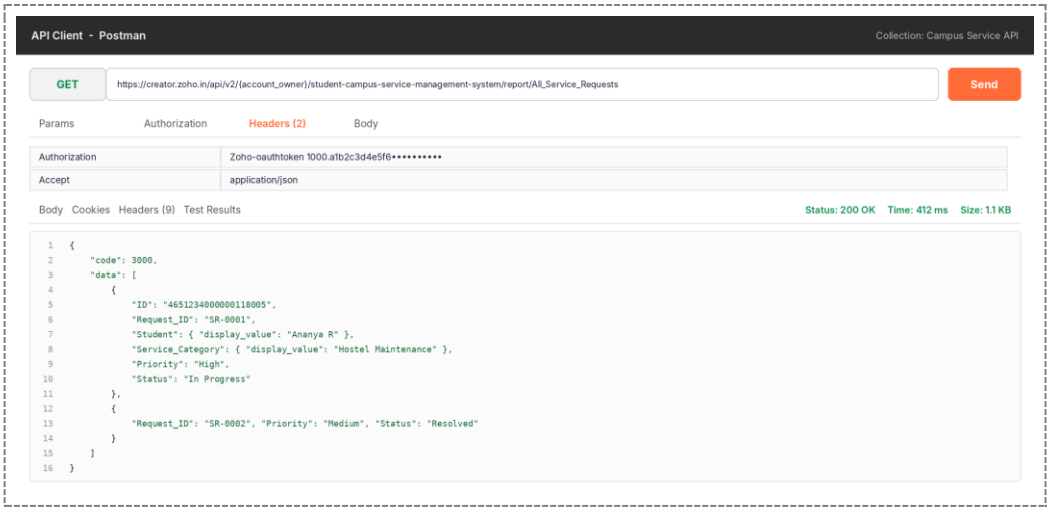


Figure 5.10: Postman Request and JSON Response from the Creator REST API

Code 5.4: Zoho Cliq Integration Script (DS-05)

```
// DS-05: High priority alert to Zoho Cliq using a secured connection
if(input.Priority == "High")
{
    payload = Map();
    payload.put("text", "High priority request " + input.Request_ID +
": " + input.Subject);
    response = invokeurl
    [
        url
:"https://cliq.zoho.in/api/v2/channelsbyname/campusservices/message"
        type :POST
        parameters :payload.toString()
        connection :"cliq_alert_conn"
    ];
    info response;
}
```

5.8 User Management and Role-Based Access

Four roles are arranged in a hierarchy: Administrator, Service Manager, Service Staff and Student (Portal). Each user is given a profile that decides which forms, reports and pages are visible and which actions are allowed. The Administrator can manage all records and configuration; the Service Manager can add and edit operational records and view the dashboard; Service Staff can view and update only the requests assigned to them and record payments; and students can add and view only their own requests, payments and feedback. Record-level criteria compare the logged-in user with the Student record, so one student cannot open another student's data. Test accounts for each role were used to verify these permissions.

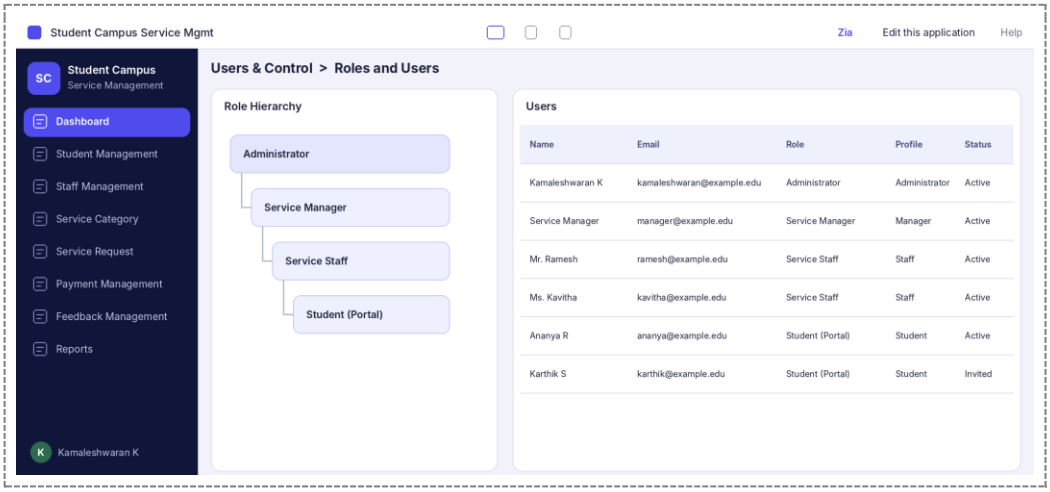


Figure 5.11: Roles and Users Configuration



Figure 5.12: Permission Profile Configuration for Service Staff

5.9 AI and Analytics Features

Zia AI features are used in the Feedback Management form. A Sentiment Analysis field reads the Comment text and records whether it is positive, neutral or negative, and a Keyword Extraction field lists the main topics mentioned. The fields were created with the help of the Zia builder assistant and then reviewed and tested before being saved. Administrators can filter the Feedback Summary report by sentiment to find services that need attention.

Analytics are provided by the dashboard charts and the Staff Workload summary report. They show request counts by category and status, the monthly trend of requests, staff workload and fee collection. These are descriptive analytics; no predictive model is claimed.

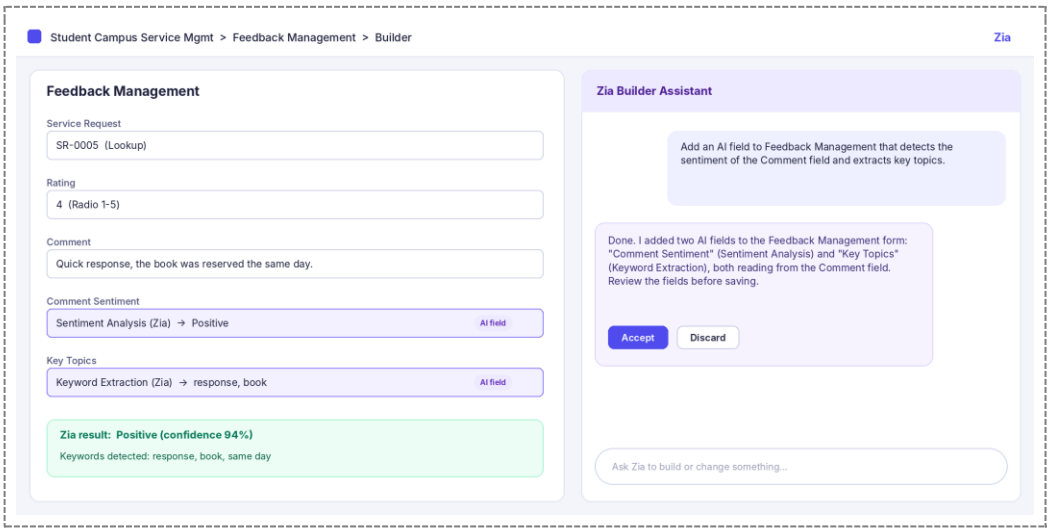


Figure 5.13: Zia AI Fields and Builder Assistant in Feedback Management

5.10 Pro-Code Extension

No pro-code extension such as a Catalyst function, a custom widget or an external microservice was needed. Every requirement was met with forms, workflows, Deluge scripts and connections, which keeps the application easy to maintain by campus staff who are not professional developers. If a future requirement cannot be met within Creator, such as complex payment verification, a Zoho Catalyst function can be called through the same connection mechanism.

5.11 Mobile and Portal Access

Students use a customer portal with their own sign-in to submit requests and view their status. Staff and administrators can also use the Zoho Creator mobile app. The same forms and reports adapt to the smaller screen, and the permissions defined for each role apply unchanged on every device.

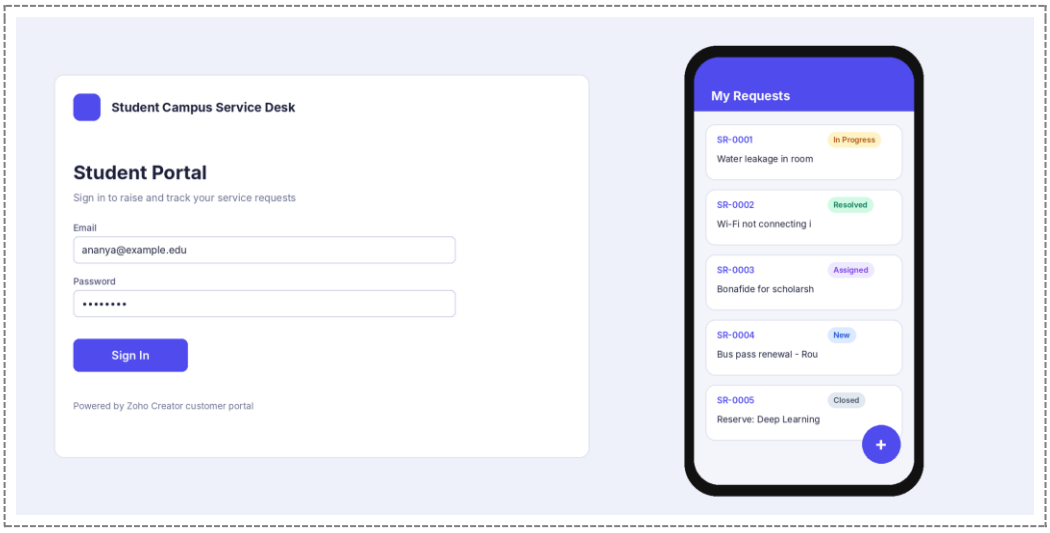


Figure 5.14: Student Portal Sign-In and Mobile View of My Requests

CHAPTER 6
TESTING AND DEBUGGING

6.1 Testing Strategy

Testing was planned at three levels. Form-level tests check required fields, valid data types, date rules and duplicate handling. Integration tests check lookup relationships, workflow triggers, emails, the Cliq alert and the API call. Acceptance tests follow a complete service request lifecycle with users in each role. Each test case in Table 6.1 is linked to a functional requirement.

6.2 Test Cases

Testing was carried out in a controlled sequence: create valid records, test invalid field values, verify lookup relationships, trigger workflows, inspect reports and the dashboard, and verify role permissions with separate accounts.

Table 6.1: Test Cases

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-01	FR-01	Add a student with all mandatory fields	Enter valid name, email, department	Student record saved	Saved as expected	Pass
TC-02	FR-01	Add a student without an email	Leave Email blank	Validation prevents submission	Message shown; not saved	Pass
TC-03	FR-01	Add a student with a duplicate email	Re-use an existing email	Duplicate rejected	Duplicate rejected	Pass
TC-04	FR-03	Submit a request through the portal	Select category, enter subject and priority	Request saved with status New	Saved as New	Pass
TC-05	FR-04	Check due-date calculation	Category SLA = 3 days, request date 24-Oct-2026	Due date 27-Oct-2026	Due date 27-Oct-2026	Pass
TC-06	FR-04	Category without SLA days	Submit request for such a category	Submission blocked with alert	Alert shown	Pass
TC-07	FR-07	Acknowledgement email	Submit a request for a student with a test email	Email received	Email received	Pass

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-08	FR-05	Assign staff	Set Assigned Staff and Status Assigned	Staff receives email	Email received	Pass
TC-09	FR-07	Resolution email	Change Status to Resolved	Student receives feedback link	Email received	Pass
TC-10	FR-08	High-priority alert	Submit a High-priority request	Message appears in Cliq channel	Message posted	Pass
TC-11	FR-08	Overdue escalation	Set due date in the past and run schedule	Administrator notified	Email received	Pass
TC-12	FR-09	Sentiment analysis	Submit a positive comment	Sentiment = Positive	Positive	Pass
TC-13	FR-11	Student record isolation	Sign in as a student and open the request report	Only own requests visible	Only own requests	Pass
TC-14	FR-11	Staff delete permission	Sign in as staff and try to delete a request	Delete option unavailable	Not available	Pass

6.3 Debugging Deluge Scripts

Scripts were debugged by adding info statements that print intermediate values, such as the SLA days and the calculated due date, and by reading the output in the script editor. Workflow executions were also checked in the workflow log, and API calls were tested separately in Postman before being used in scripts. Figure 6.1 shows the due-date script with its info output.



Figure 6.1: Deluge Debug Output for the Due-Date Script

6.4 Defect Log

Table 6.2: Defect Log

Bug ID	Description	Root Cause	Fix Applied	Status
BUG-01	Due date was empty for a new category	SLA Days was left blank in the category record	Made SLA Days mandatory and added the check in DS-01	Closed
BUG-02	Staff could see all requests in the report	Profile had no record-level criteria	Added criteria: Assigned Staff is the logged-in user	Closed
BUG-03	Resolution email sent again on every edit	Workflow ran for every update	Added condition: Status is Resolved	Closed

6.5 User Acceptance Testing

User acceptance testing was done with test accounts for each role. Defects found were added to the defect log, fixed and retested before the application was published. The feedback collected is summarised in Table 6.3.

Table 6.3: User Acceptance Feedback

User	Role Tested	Feedback	Action Taken
Test user 1	Student	Form is simple; status of requests is easy to see.	Added coloured status labels in the report
Test user 2	Service Staff	Only assigned requests are shown, which is clear.	None required
Test user 3	Administrator	Dashboard gives a quick summary of open requests.	Added quick action links

CHAPTER 7
DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

The application was developed in the Development environment, promoted to Stage for acceptance testing and finally published to Production. Changes are always made in Development first, so the live application used by students is not disturbed. Figure 7.1 shows the three environments with their versions, and Table 7.1 lists the checks completed before publishing.

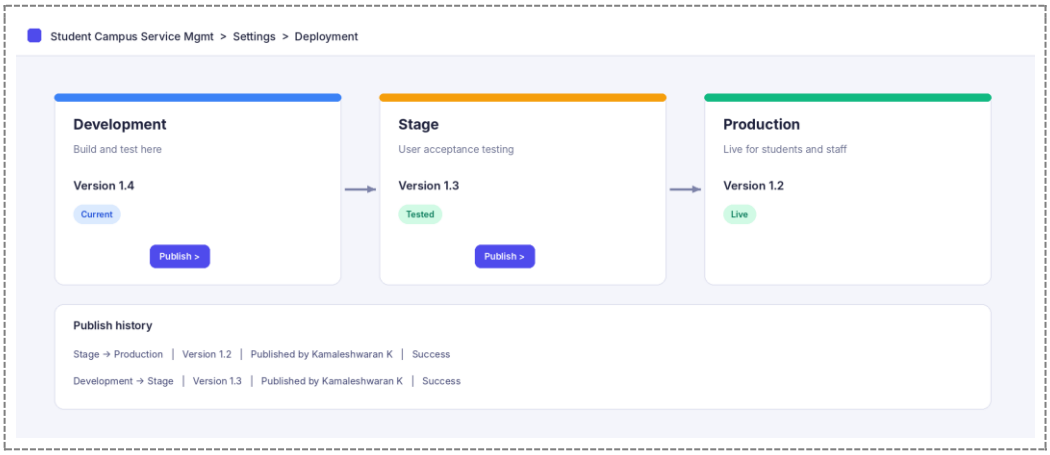


Figure 7.1: Environments and Publish to Production

Table 7.1: Deployment Checklist

S.No.	Checklist Item	Done (✓)
1	All test cases executed in Development / Stage	✓
2	Sample/test data removed or isolated	✓
3	Role permissions tested with separate accounts	✓
4	Email delivery tested with authorised addresses	✓
5	Connections authorised with minimum scopes	✓
6	Reports and dashboard reconciled with records	✓
7	Backup / export procedure reviewed	✓
8	Application published to Production	✓

7.2 Performance Optimisation

Performance was kept acceptable by design choices rather than by heavy tuning. Scripts fetch only the single student or category record that they need, reports use filters and criteria to open focused views, and the schedule processes only records that are overdue. Table 7.2 lists the measures applied. Timing figures are not claimed because the sample dataset is small.

Table 7.2: Performance Optimisation Measures

Optimisation Applied	Component	Approach	Expected Benefit
Criteria-based fetch	DS-01 to DS-04	Fetch by record ID instead of looping over all records	Fewer records read per submission
Report filters and criteria	Open Requests, Pending Payments	Show only records needed by the role	Faster loading of focused views
Criteria in schedule	DS-06	Select only overdue, unresolved requests	Shorter nightly processing
Dashboard from reports	Campus Service Dashboard	Charts use existing report criteria	No duplicate calculations

7.3 Security Features

Table 7.3: Security Controls Implemented

Security Control	Where Applied	Threat Mitigated
Role-based permissions	Administrator, Manager, Staff and Student profiles	Unauthorised access to records and actions
Record-level criteria	Student and Staff views of requests	One user reading another user's data
Mandatory fields and validation	Student, request, payment and feedback forms	Incomplete or invalid data
OAuth connections with minimum scopes	cliq_alert_conn, creator_api_conn	Exposure of passwords or excessive API access
Recipient checks	Email workflows	Sending messages to empty or invalid recipients
Hosted over HTTPS	Zoho Creator service	Interception of data in transit
Audit trail	Administration settings	Untraceable configuration or data changes

Security controls follow the least-privilege principle: each role is given only the access that its tasks require. Student contact information is shown only to authorised staff, and

email messages contain only the information needed for the student to identify the request.

7.4 Governance and Administration

Governance practices include meaningful form and field names, restricted access to student contact information, review of workflow changes before publishing, periodic export of data as a backup and regular review of the audit trail. Figure 7.2 shows the audit trail listing who added or changed records and configuration.



Figure 7.2: Audit Trail and Administration Settings

CHAPTER 8
RESULTS AND DISCUSSION

8.1 Application Output

A sample end-to-end scenario begins when a student signs in to the portal and submits a high-priority hostel maintenance request (Figure 8.1). The system sets the due date from the category SLA, sends an acknowledgement email and posts a Zoho Cliq alert. The administrator then assigns the request to the warden, who receives an assignment email (Figure 8.2). When the warden marks the request as Resolved, the student receives a resolution email with a feedback link (Figure 8.3), and the feedback comment is analysed by Zia.

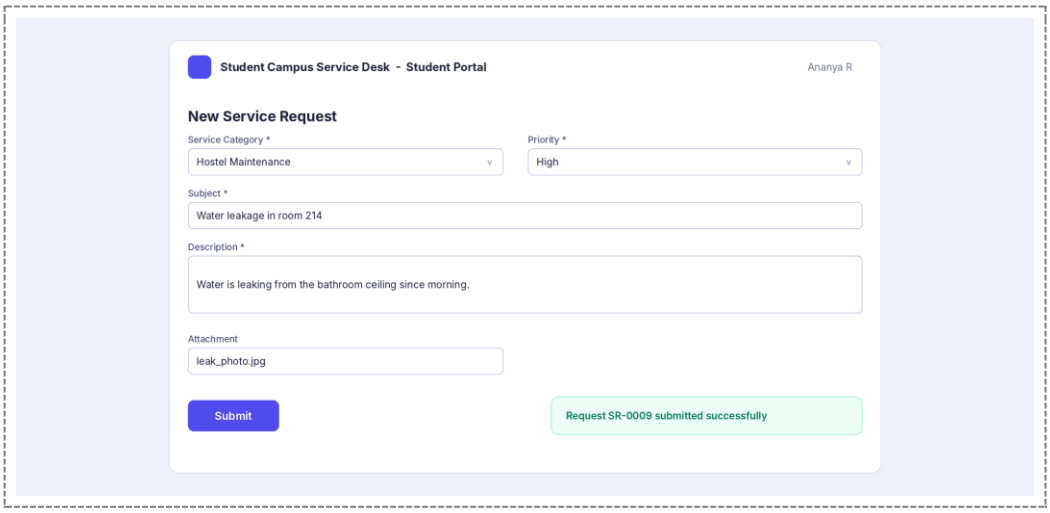


Figure 8.1: Scenario Step 1 – Student Submits a Service Request in the Portal

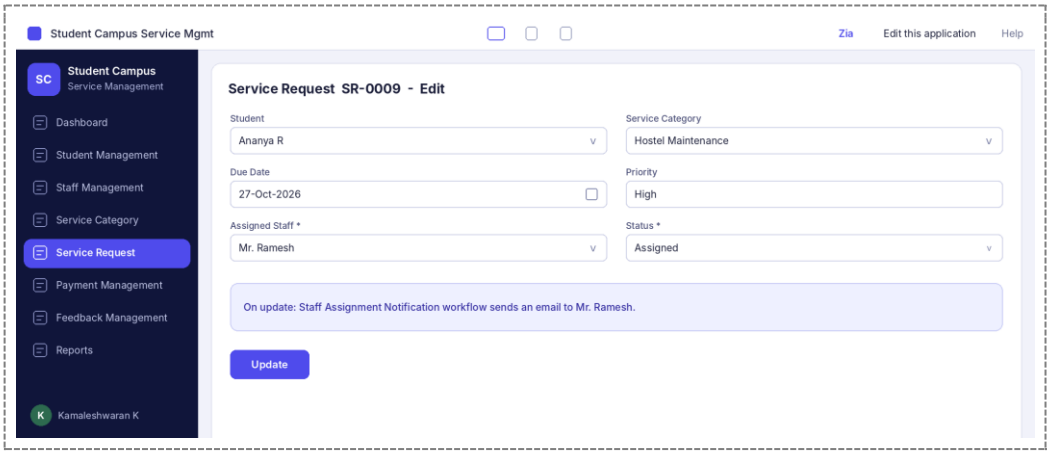


Figure 8.2: Scenario Step 2 – Administrator Assigns the Request to Staff

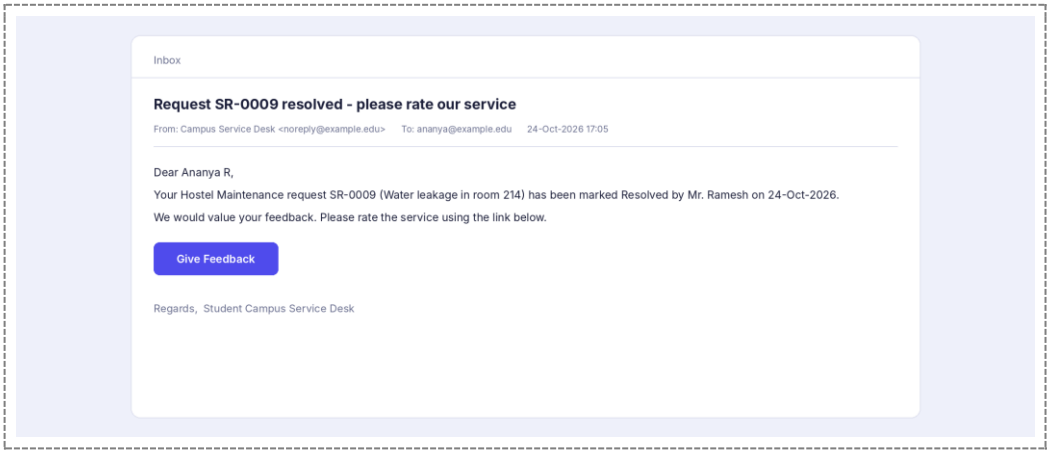


Figure 8.3: Scenario Step 3 – Resolution Email Received by the Student

The scenario demonstrates the full request lifecycle: register, acknowledge, assign, resolve and collect feedback, with the dashboard and reports updating automatically at each step.

8.2 Achievement of Objectives

Table 8.1: Mapping of Objectives to Outcomes

Objective (Sec. 1.3)	Implemented Through	Outcome / Evidence
Create related records for campus services	Sections 4.2 and 5.2	Six forms with lookup relationships (Figs. 4.2, 5.2–5.4).
Automate validation, due dates and notifications	Sections 5.5 and 5.6	Scripts DS-01 to DS-06 and eight workflows tested in Table 6.1.
Provide reports and dashboard	Sections 5.3–5.4	Six reports, summary tiles and four charts (Figs. 5.5–5.7).
Integrate with Zoho Cliq / REST API and use Zia	Sections 5.7 and 5.9	Two OAuth connections, Cliq alert, Postman response and Zia fields (Figs. 5.9, 5.10, 5.13).
Apply role-based access	Sections 5.8 and 7.3	Four roles, profiles and record-level criteria (Figs. 5.11, 5.12).
Deploy securely	Sections 7.1 and 7.4	Development, Stage and Production environments; audit trail (Figs. 7.1, 7.2).

The application consolidates the main records needed for campus service handling and provides linked forms, reports, a dashboard, validation logic, automated notifications, integrations, AI-assisted feedback analysis and role-based access.

8.3 Comparison with the Existing System

Table 8.2: Existing System versus Proposed Application

Parameter	Existing System	Proposed Application	Improvement
Request registration	Paper slip or register entry at the office	Online form through portal, web or mobile	Requests can be raised any time and are never lost
Status visibility	Student must visit or call	Student sees status in My Requests	Transparent tracking
Deadlines	Not defined or tracked	Due date from SLA; daily overdue escalation	Delays are detected automatically
Notifications	Manual calls or messages	Workflow emails and Cliq alerts	Faster communication
Reporting	Manual counting from registers	Reports, dashboard and charts	Instant summaries for management
Feedback	Rarely collected	Rating, comment and Zia sentiment	Service quality can be measured

Compared with a register-and-spreadsheet process, the proposed application centralises related records and provides reusable reports and automation. The improvement is qualitative; no time-saving percentage is claimed because a before-and-after timing study was not performed.

8.4 Limitations of the Low-Code Approach

The application depends on Zoho Creator's supported features and the account edition. Complex integrations, advanced payment verification, extensive custom user interfaces and very large data volumes may need pro-code extensions. Platform limits on API calls and storage must also be considered, and the application cannot be moved easily to another platform.

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

The project illustrates the no-code, low-code and pro-code spectrum. Visual form, report and dashboard builders provide much of the application structure without code, Deluge scripts and connections add custom logic and integration, and pro-code tools such as Catalyst remain available for needs beyond the platform. Choosing the lowest level that satisfies a requirement kept the project fast to build and simple to maintain.

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

The Student Campus Service Management System brings student, staff, service category, request, payment and feedback data into one Zoho Creator application. It uses lookup relationships, validation, Deluge scripts, workflows, an approval process and a Blueprint to manage the full life of a service request. Reports, a dashboard and charts give administrators clear visibility, while connections to Zoho Cliq and the Creator REST API, Zia AI fields and role-based access extend the application beyond simple data entry. The project shows that a practical, secure and automated campus service desk can be built quickly with a low-code platform.

9.2 Future Enhancements

- Enhancement 1 – Online payment-gateway integration with automatic receipt generation for chargeable services.
- Enhancement 2 – Push notifications and WhatsApp or SMS alerts for status changes.
- Enhancement 3 – Integration with the college ERP for automatic student data import.
- Enhancement 4 – Predictive analytics for expected request volumes and staff planning using Zoho Analytics.

9.3 Learning Outcomes and Career Relevance

The project develops practical skills in relational data modelling through lookup fields, low-code form and report design, Deluge scripting, workflow automation, secure API integration, role-based access, AI-assisted features, testing and deployment. These skills are directly relevant to roles in business application development, low-code engineering and enterprise automation.

REFERENCES

[1] Zoho Corporation, “Zoho Creator Help Documentation.” [Online]. Available: https://www.zoho.com/creator/help/. Accessed: 24 Oct. 2026.

[2] Zoho Corporation, “Deluge Help Documentation.” [Online]. Available: https://www.zoho.com/deluge/help/. Accessed: 24 Oct. 2026.

[3] Zoho Corporation, “Zoho Creator Workflows.” [Online]. Available: https://www.zoho.com/creator/help/workflows/. Accessed: 24 Oct. 2026.

[4] Zoho Corporation, “Zoho Creator Reports and Dashboards.” [Online]. Available: https://www.zoho.com/creator/help/reports/. Accessed: 24 Oct. 2026.

[5] Zoho Corporation, “Zoho Creator REST API v2.” [Online]. Available: https://www.zoho.com/creator/help/api/v2/. Accessed: 24 Oct. 2026.

[6] OWASP Foundation, “Application Security Verification Standard (ASVS).” [Online]. Available: https://owasp.org/www-project-application-security-verification-standard/. Accessed: 24 Oct. 2026.

[7] A. Sahay, A. Indamutsa, D. Di Ruscio and A. Pierantonio, “Supporting the understanding and comparison of low-code development platforms,” in Proc. 46th Euromicro Conf. on Software Engineering and Advanced Applications (SEAA), 2020, pp. 171–178.

[8] M. Fowler, Patterns of Enterprise Application Architecture. Boston, MA, USA: Addison-Wesley, 2002.

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

Table A.1: Feature Coverage Checklist

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
I	Application created & sections/menu organised	Y	Student Campus Service Management System	5.1	Fig. 5.1
II	Forms with ≥ 8 distinct field types	Y	Student, Service Request, Payment and Feedback forms	5.2	Figs. 5.2–5.4
II	Lookup relationships between forms	Y	Student, category, staff and request lookups	4.2, 5.2	Fig. 4.2
II	Subform where applicable	N	Not required; payments and feedback are separate linked forms	4.2	Justified in 4.2
II	Reports with filtering and sorting	Y	Six reports (List, Kanban, Summary)	5.3	Fig. 5.5
II	Dashboard / page with summaries	Y	Campus Service Dashboard	5.4	Figs. 5.6–5.7
II	Deluge validation / calculation scripts	Y	DS-01 validation and due-date calculation	5.5	Code 5.1
II	At least three meaningful workflows	Y	Five form workflows, schedule, approval, Blueprint	5.6	Fig. 5.8
II	Email notifications tested end-to-end	Y	Acknowledgement, assignment, resolution emails	6.2, 8.1	Fig. 8.3
III	Roles and permissions	Y	Four roles with profiles	5.8, 7.3	Figs. 5.11–5.12
IV	External integration / API	Y	Zoho Cliq alert via connection	5.7	Fig. 5.9, Code 5.4
V	Security and governance	Y	Permissions, validation, audit trail	7.3–7.4	Fig. 7.2
II	Form workflow	Y	DS-01 to DS-05	5.6	Fig. 5.8
II	Scheduled workflow	Y	Overdue Request Escalation (daily 9:00 AM)	5.6	Code 5.3
II	Approval process	Y	Certificate Approval	5.6	Table 5.3
II	Blueprint	Y	Request Lifecycle	5.6	Table 5.3

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
II	Batch / report workflow	N	Not required for this scope	5.6	Justified in 5.6
II	Testing & debugging evidence	Y	Test cases and debug output	6	Table 6.1, Fig. 6.1
III	Integration with a Zoho service	Y	Zoho Cliq	5.7	Fig. 5.9
III	Connection / REST API call (invokeurl)	Y	DS-05 using cliq_alert_conn	5.7	Code 5.4
III	Webhook or Creator REST API	Y	Creator REST API v2 tested in Postman	5.7	Fig. 5.10
III	Users, roles and permission profiles	Y	Roles, users and profiles	5.8	Figs. 5.11–5.12
III	Record-level access criteria / customer portal	Y	Student portal with own-record criteria	5.8, 5.11	Fig. 5.14
III	Zia / AI feature	Y	Sentiment and keyword AI fields	5.9	Fig. 5.13
III	Analytics (charts / Zoho Analytics)	Y	Dashboard charts (Zoho Analytics not used)	5.4, 5.9	Fig. 5.7
IV	Performance optimisation applied & measured	Y	Measures applied; timings not claimed	7.2	Table 7.2
IV	Deployment across environments	Y	Development, Stage, Production	7.1	Fig. 7.1
IV	Security controls (encryption, OAuth, permissions)	Y	OAuth connections, HTTPS, profiles	7.3	Table 7.3
IV	Governance (audit trail, administration)	Y	Audit trail review	7.4	Fig. 7.2
V	Pro-code extension (Catalyst / widgets / custom functions)	N	Not required; justified in 5.10	5.10	—
V	Low-code limitations & spectrum analysis	Y	Sections 8.4 and 8.5	8.4, 8.5	—
—	Mobile app / portal access	Y	Student portal and mobile app	5.11	Fig. 5.14

Total features implemented: 29 / 32

APPENDIX B – COURSE OUTCOME MAPPING

Table B.1: Mapping of Course Outcomes to Report Chapters

CO	Course Outcome Statement	Evidenced In	Self-Assessed Level	Examiner Level
CO1	Explain the principles, architecture, and business value of low-code application platforms.	Chapters 1–2	3	
CO2	Develop business applications using forms, reports, workflows and Deluge.	Chapters 4–6	3	
CO3	Apply integration, access control, security and deployment concepts.	Chapters 5 and 7	3	
CO4	Test, debug and document a low-code application.	Chapters 6 and Appendices	3	
CO5	Evaluate the application and identify practical future enhancements.	Chapters 8–9	3	

APPENDIX C – COMPLETE DELUGE SOURCE CODE

C.1 Service Request – Validation and Due-Date Script (DS-01)

```
// Script ID : DS-01
// Location  : Service Request > Workflow > On Add > On Validate
// Purpose   : Validate request date and SLA days, then set the due
date

cat = Service_Category[ID == input.Service_Category];
if(input.Request_Date == null)
{
    alert "Request Date is required";
    cancel submit;
}
if(cat.SLA_Days == null || cat.SLA_Days < 1)
{
    alert "SLA days are not defined for this service category";
    cancel submit;
}
input.Due_Date = input.Request_Date.addDay(cat.SLA_Days.toLong());
```

C.2 Service Request – Acknowledgement Email Script (DS-02)

```
// Script ID : DS-02
// Location  : Service Request > Workflow > On Add > On Success
// Purpose   : Email the student that the request is registered

stu = Student_Management[ID == input.Student];
cat = Service_Category[ID == input.Service_Category];
if(stu.Email != "")
{
    sendmail
    [
        from :zoho.adminuserid
        to :stu.Email
        subject :"Request " + input.Request_ID + " received"
        message :"Dear " + stu.Student_Name + ", your " +
cat.Category_Name + " request is registered. Due date: " +
input.Due_Date
    ]
}
```

C.3 Service Request – Staff Assignment Notification (DS-03)

```
// Script ID : DS-03
// Location  : Service Request > Workflow > On Edit > On Success
// Purpose   : Notify the assigned staff member

if(input.Assigned_Staff != null && input.Status == "Assigned")
{
    staff = Staff_Management[ID == input.Assigned_Staff];
    sendmail
    [
        from :zoho.adminuserid
        to :staff.Email
        subject :"New request assigned: " + input.Request_ID
        message :"Request " + input.Request_ID + " (" + input.Subject
+ ") is assigned to you. Due date: " + input.Due_Date
    ]
}
```

C.4 Service Request – Resolution and Feedback Invitation (DS-04)

```
// Script ID : DS-04
// Location  : Service Request > Workflow > On Edit > On Success
// Purpose   : Tell the student the request is resolved and invite
feedback

if(input.Status == "Resolved")
{
    stu = Student_Management[ID == input.Student];
    if(stu.Email != "")
    {
        sendmail
        [
            from :zoho.adminuserid
            to :stu.Email
            subject :"Request " + input.Request_ID + " resolved -
please rate our service"
            message :"Dear " + stu.Student_Name + ", your request has
been resolved. Please give feedback in the student portal."
        ]
    }
}
```

C.5 Service Request – High Priority Zoho Cliq Alert (DS-05)

```
// Script ID : DS-05
// Location  : Service Request > Workflow > On Add > On Success
// Purpose   : Post an alert in Zoho Cliq using the cliq_alert_conn
connection

if(input.Priority == "High")
{
    payload = Map();
    payload.put("text", "High priority request " + input.Request_ID +
": " + input.Subject);
    response = invokeurl
    [
        url
:"https://cliq.zoho.in/api/v2/channelsbyname/campusservices/message"
        type :POST
        parameters :payload.toString()
        connection :"cliq_alert_conn"
    ];
    info response;
}
```

C.6 Schedule – Overdue Request Escalation (DS-06)

```
// Script ID : DS-06
// Location  : Schedule > Overdue Request Escalation (daily 9:00 AM)
// Purpose   : Email the administrator about overdue, unresolved
requests

for each rec in Service_Request[Due_Date < zoho.currentdate && Status
!= "Resolved" && Status != "Closed"]
{
    sendmail
    [
        from :zoho.adminuserid
        to :zoho.adminuserid
        subject :"Overdue request " + rec.Request_ID
        message :"Request " + rec.Request_ID + " (" + rec.Subject + ")
is past its due date " + rec.Due_Date
    ]
}
```

APPENDIX D – USER MANUAL

D.1 Accessing the Application

1. Open the Zoho Creator application link provided by the administrator and sign in with your own account.

2. Use the left navigation menu to open the forms and reports allowed for your role.

D.2 Instructions for Students

1. Sign in to the student portal and open Service Request.

2. Select the service category, enter a subject and description, choose the priority and attach a file if needed.

3. Click Submit. A confirmation email is sent and the request appears in My Requests with its status and due date.

4. When the request is resolved, open the link in the resolution email and give a rating and comment in Feedback.

D.3 Instructions for Service Staff

1. Sign in and open the All Service Requests report; only requests assigned to you are shown.

2. Open a request, change the status to In Progress and add notes when work starts.

3. When the work is complete, change the status to Resolved so that the student is notified.

4. For chargeable services, open Payment Management and record the payment against the request.

D.4 Instructions for the Administrator

1. Maintain students, staff and service categories (including SLA days and fees) in their forms.

2. Open the Open Requests report, assign staff and set the priority; certificate requests wait for manager approval.

3. Review the Campus Service Dashboard daily for open, overdue and high-priority requests.

4. Manage users, roles and profiles from Users & Control, and review the audit trail when needed.

APPENDIX E – EVALUATION RUBRIC

(For Examiner Use Only)

Student Name	KAMALESHWARAN K	Register No.	2116241501079
Project Title	Student Campus Service Management System	Date	

Table E.1: Evaluation Rubric for the Individual Project

S.No.	Criterion	Indicators	Max. Marks	Marks Awarded
1	Problem Definition & Requirement Analysis	Clarity of problem, measurable objectives, complete FR/NFR, use cases	10	
2	System Design	Architecture, ER/data model, form specifications, workflow design, permission matrix	10	
3	Forms, Reports & Pages	Variety of field types, lookups/subforms, report types, UI Builder dashboards	15	
4	Deluge Scripting & Workflows	Correctness and depth of scripts; form, schedule, approval, Blueprint, batch workflows	15	
5	Integration, RBAC & AI	Zoho/external integrations, connections, roles & permissions, Zia/analytics	10	
6	Testing, Deployment, Security & Governance	Test coverage, environments, optimisation, security controls, audit	10	
7	Live Demonstration	End-to-end working of the application across roles; feature checklist verified	15	
8	Project Report	Adherence to template and format, clarity, figures, references	5	
9	Viva-Voce	Understanding of the platform, design decisions and low-code concepts	10	
	TOTAL		**100**	

Table E.2: Performance Level Descriptors

Level	% of Max. Marks	Descriptor
Excellent	90–100	Feature implemented completely, correctly and beyond the basic requirement; student explains design choices confidently.
Good	70–89	Feature implemented correctly with minor gaps; explanation adequate.
Satisfactory	50–69	Feature partially implemented or works only for the main path; limited explanation.
Needs Improvement	< 50	Feature missing, non-functional or not understood by the student.

Signature of Internal Examiner

Name: _____

Signature of External Examiner

Name: _____

APPENDIX F – VIVA-VOCE RECORD SHEET

(For Examiner Use Only)

Student Name	KAMALESHWARAN K	Register No.	241501079
Department / Section	AIML	Date of Viva	
Application URL Verified	Yes / No	Demo Duration	_____ min

Table F.1: Viva-Voce Questions and Responses

Q.No.	Question / Topic Asked	Unit	Response (E / G / S / NI)	Remarks
1				
2				
3				
4				
5				
6				
7				
8				

E – Excellent G – Good S – Satisfactory NI – Needs Improvement

Table F.2: Marks Summary

Component	Max. Marks	Marks Awarded
Criteria 1–8 (Appendix E)	90	
Viva-Voce	10	
TOTAL	**100**	

Examiners' Remarks:

_____

_____

INTERNAL EXAMINER

Name: _____

EXTERNAL EXAMINER

Name: _____